



Machine Learning for Petroleum Engineers Using Python

Módulo 3 · Clase 1

Regresión Lineal: Tu Primer Modelo Predictivo



Carlos Enrique Mosquera Trujillo



SLB Ecuador

Empieza el Módulo de Machine Learning

✓ Hasta ahora:

- ✓ Módulo 1: Python y carga de datos (pandas, CSV, LAS)
- ✓ Módulo 2: exploración y limpieza (EDA)
- ✓ Sabemos **describir** los datos: media, correlación, gráficos

*Describir contesta “¿qué pasó?”. Hoy damos el salto a “¿qué **va** a pasar?”.*

▶▶ Hoy (Clase 1 de ML):

- Qué es **predecir** y qué es un **modelo**
- La **regresión lineal**: la recta que mejor se ajusta
- Cómo medir si el modelo es **bueno** (R^2 , RMSE)
- Muchas variables, estandarización y **train/test**

*Reto real: **reconstruir un registro de pozo** a partir de otros.*

¿Qué es Machine Learning?

Un cambio de enfoque: en vez de escribir la regla, dejamos que el computador la aprenda de los ejemplos.

</> Programación clásica

El humano escribe la regla:

```
si presion > 3000: alerta
```

Funciona cuando la regla **se conoce** y se puede escribir a mano.

🤖 Machine Learning

Le damos **ejemplos con la respuesta** y el computador **encuentra la regla** que mejor los explica.

Funciona cuando la regla es demasiado compleja para escribirla a mano.

- Hoy hacemos aprendizaje **supervisado**: aprender de ejemplos que **ya traen la respuesta correcta**.
- Y dentro de lo supervisado, **regresión**: la respuesta que buscamos es **un número** (el valor del sónico).

El Problema Real de Hoy: un Sensor que Falla

El sensor que mide el registro sónico se dañó a mitad del pozo. Hay un tramo sin datos.

- \$ Volver a correr la herramienta significa traer de nuevo la unidad de registro y **parar el pozo**: días de equipo y **cientos de miles de dólares**.
- ⚠ A veces ni siquiera se puede: el pozo ya está entubado.
- 💡 **Decisión de negocio:** ¿pagamos por re-loguear, o **confiamos en un modelo** que reconstruya el sónico a partir de los sensores que **sí** funcionaron?

La apuesta:

aprender de los tramos donde **sí** tenemos las tres medidas (sónico, densidad, porosidad) para **rellenar** donde el sensor falló — sin gastar en re-loguear.

Es un problema real de la industria; hay papers prediciendo el sónico del campo Volve exactamente así.

La Gran Idea

Relaciones entre variables que podemos modelar

Ustedes Ya Piensan en Relaciones

Nadie llega en blanco: en la operación diaria ustedes ya predicen usando relaciones.

- ↓ A más **profundidad** → más **presión** y más **temperatura**.
- 🕒 Más **horas de bomba** → más **consumo de diésel**.
- 💧 Más **agua** en el fluido → más **costo de tratamiento**.

“Si el pozo está a 4 000 m, espera unos 120 °C.”

Eso es una **predicción** — hecha de memoria, con una **relación entre dos variables** que ustedes ya conocen.

Hoy haremos exactamente eso mismo, pero **preciso, repetible y auditable**.

¿Qué es una Relación entre Dos Variables?

Una variable es algo que medimos y que cambia. Hay relación cuando conocer una nos dice algo de la otra.

✓ Sí hay relación

Profundidad y temperatura. Si me dicen “4 000 m”, ya **no** espero *cualquier* temperatura: espero una zona estrecha de valores. Saber una me **acerca** a la otra.

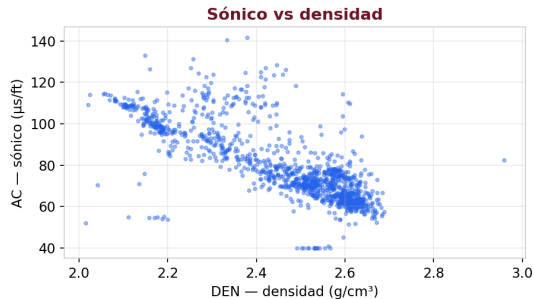
✗ NO hay relación

Día de la semana y densidad de la roca. Saber que hoy es martes no me acerca **en nada** al valor de la densidad. Conocer una no dice nada de la otra.

💡 **Test de bolsillo:** ¿saber A me ayuda a adivinar B? Si la respuesta es sí, hay una relación que podemos **aprovechar**.

La Relación de Hoy, Dibujada

Dos mediciones del pozo: densidad (DEN) y sónico (AC). Un punto por profundidad.

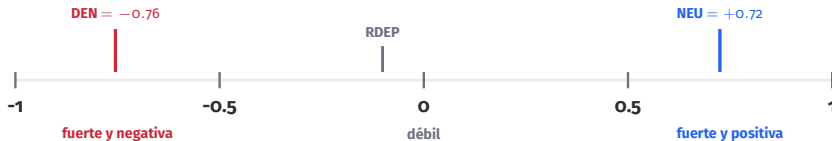


- Cada punto es **una profundidad** del pozo (hay 6 893).
- La nube **no es redonda: está inclinada**. Inclinada = hay relación.
- Léanla: roca más densa → el sonido viaja más rápido → sónico más bajo.
- ? La pregunta del día: ¿es **lo bastante firme** para rellenar el tramo del sensor dañado?

Ponerle Número a la Relación: la Correlación

La correlación responde dos preguntas con un solo número entre -1 y $+1$.

1. **¿En qué dirección va?** El **signo**: con $+$, cuando una crece la otra tiende a crecer; con $-$, cuando una crece la otra tiende a caer.
2. **¿Qué tan firme es?** La **cercanía a ± 1** : cerca de ± 1 la nube es casi una línea (saber una casi fija la otra); cerca de 0 , la nube es redonda (saber una no ayuda).



💡 Regla de bolsillo del curso: $|r| > 0.7$ fuerte · $0.3-0.7$ media · < 0.3 débil. Nuestra pareja DEN-AC da -0.76 : fuerte y negativa.

¿Qué Significa que una Relación sea Lineal?

Lineal = la relación es pareja: el mismo paso en una variable produce siempre el mismo paso en la otra.

- \$ Tanquear el carro:** el galón 1 cuesta lo mismo que el galón 10. Por eso el total es $\text{precio} \times \text{galones}$: **una recta.**
- ↓ Presión hidrostática:** cada 100 m de profundidad suman $\approx$ la misma presión, arriba o abajo del pozo.
- ✓** Si el paso vale **siempre lo mismo**, el dibujo de la relación es una **línea recta.**



💡 A ese “precio del paso” lo llamaremos **pendiente** (m). Es el corazón de la recta.

La Relación, Escrita como Fórmula

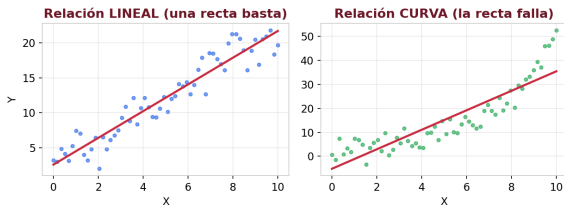
Si el paso siempre vale lo mismo, la relación completa cabe en una línea de álgebra.

$$\underbrace{AC}_{\text{predicción}} = \underbrace{b}_{\text{intercepto (dónde arranca)}} + \underbrace{m}_{\text{pendiente (precio del paso)}} \times \underbrace{DEN}_{\text{dato conocido}}$$

- m (**pendiente**): cuánto cambia el sónico por cada +1 de densidad.
- b (**intercepto**): el punto de arranque de la recta.
- ✓ **“Entrenar el modelo”** = encontrar los mejores m y b para nuestros datos. Eso lo hace el computador en un segundo.

No Todas las Relaciones Son Parejas

En el mundo real, muchas relaciones existen y son claras... pero no son rectas.



- **Declive de un pozo:** el primer año cae muchísimo; el quinto, casi nada. El paso **no** vale lo mismo.
- **Bombeo vs frecuencia:** sube al inicio y luego llega a una **meseta**: el pozo no da más.
- ⚠ En esos casos la correlación (que busca rectas) **subestima** la relación, y una recta la describe **mal**.

💡 Hoy empezamos por la **recta** porque es el modelo más simple de leer y explicar. Para lo curvo existen modelos más flexibles: los veremos en las próximas clases.

De la Relación a la Predicción

Si la relación es firme y aproximadamente recta, la fórmula se vuelve una máquina de responder.



- Un **modelo** no es más que eso: la relación **escrita como fórmula**, lista para usarse una y mil veces.
- **Machine Learning** = el computador **encuentra la fórmula** a partir de los ejemplos, en vez de que la programemos a mano.
- ✓ La usaremos para **rellenar el tramo del sensor dañado** — sin re-loguear.

Vocabulario de Machine Learning

Dos palabras que usaremos todo el módulo:



Target (objetivo)

Lo que queremos **predecir**.
Se le dice también y.

Aquí: el sónico AC.



Features (variables)

Lo que usamos **para** predecir.
Se les dice también X.

Aquí: DEN, NEU, GR, RDEP.

Un modelo aprende la relación $X \rightarrow y$ a partir de los ejemplos.

Los Datos

El well log que vamos a usar

El Dataset de Hoy: el Well Log de Volve

El mismo pozo 15/9-19 de Volve que ya conocemos, ahora como tabla para modelar.

Python

```
import pandas as pd
reg = pd.read_csv(url)
reg.head()
```

- 6 893 filas: cada una es una **profundidad** con todas sus mediciones.
- Ya está **limpio** (sin NaN) — eso lo hicieron en el Módulo 2.
- Queremos predecir la columna AC (el sónico).

>_ Output

	PROF	GR	DEN	NEU	AC
0	3550.2	55.8	2.17	51.2	54.6
1	3550.4	55.1	2.17	51.2	54.6
2	3550.5	56.1	2.17	51.2	54.6

¿Qué Mide Cada Columna? (1/2)

Antes de modelar, entendamos las variables. Todas son mediciones físicas de la roca.

Columna	Unidad	Qué mide
AC (objetivo)	$\mu\text{s}/\text{ft}$	Sónico: cuánto tarda el sonido en cruzar la roca. Roca dura $\rightarrow$ rápido $\rightarrow$ AC bajo.
DEN	g/cm^3	Densidad: qué tan pesada es la roca.
NEU	%	Neutrón: mide el hidrógeno de los fluidos $\rightarrow$ indica porosidad .

💡 Nota: en otros softwares DEN y NEU aparecen como RHOB y NPHI. Es la misma medición, distinto nombre.

¿Qué Mide Cada Columna? (2/2)

Columna	Unidad	Qué mide
GR	GAPI	Gamma Ray: radioactividad natural. Alta en arcillas, baja en arenas limpias.
RDEP	ohm·m	Resistividad: resistencia al paso de corriente. Alta = petróleo; baja = agua salada.
PROF	m	Profundidad: dónde se tomó la medición (identifica la fila).

🎯 **Target (lo que predecimos):**

AC — el sísmico

☰ **Features (lo que usamos):**

DEN, NEU, GR, RDEP

Explorar: ¿Qué Feature Predice Mejor?

Calculamos la correlación de cada feature con el target. Nos dice por dónde empezar.

Python

```
reg.corr()["AC"]
```

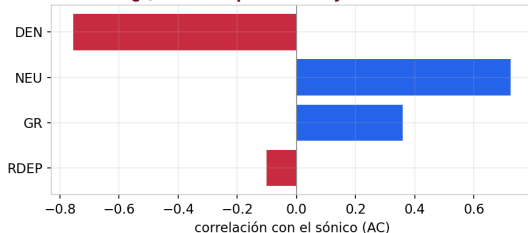
Output

```
DEN    -0.76  
NEU     0.72  
GR      0.36  
RDEP   -0.10
```

✓ DEN y NEU son las más fuertes; RDEP casi no aporta.

→ DEN negativo tiene sentido físico: roca más densa → sonido más rápido → sónico más bajo.

¿Qué curva predice mejor el sónico?

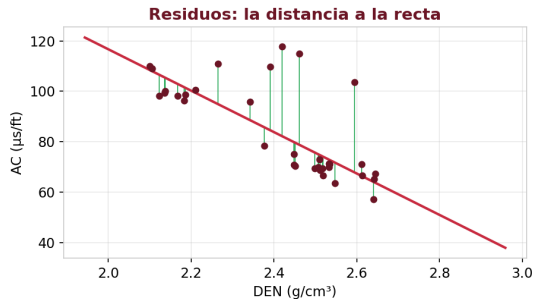


La Mejor Recta

Regresión lineal simple

¿Cuál es la “Mejor” Recta? Los Residuos

La mejor recta es la que pasa más cerca de todos los puntos.



- El **residuo** (verde) es el error: distancia entre el punto real y la recta.
- La mejor recta **minimiza** esos errores (los eleva al cuadrado y los suma: *mínimos cuadrados*).
- 🤖 `scikit-learn` encuentra esa recta óptima por nosotros.

Nuestro Primer Modelo con scikit-learn

scikit-learn (sklearn) es la librería de Machine Learning. Tres pasos y listo.

Python

```
from sklearn.linear_model import  
    LinearRegression  
  
X = reg[["DEN"]]      # feature  
y = reg["AC"]         # target  
  
modelo = LinearRegression()  
modelo.fit(X, y)      # entrenar  
  
print(modelo.intercept_, modelo.coef_)
```

>_ Output

```
280.9    [-82.1]
```

- X entre **corchetes dobles**: sklearn espera una tabla de features.
- `.fit(X, y)`: aquí el modelo **aprende** la mejor recta.
- ✓ Resultado: $b = 280.9$, $m = -82.1$.

Interpretar y Predecir

El modelo aprendido es: $AC = 280.9 - 82.1 \times DEN$

Leer el coeficiente: Por cada $+1 \text{ g/cm}^3$ de densidad, el sónico **baja 82 $\mu\text{s/ft}$** . El signo negativo confirma la física: roca más densa, sonido más rápido.

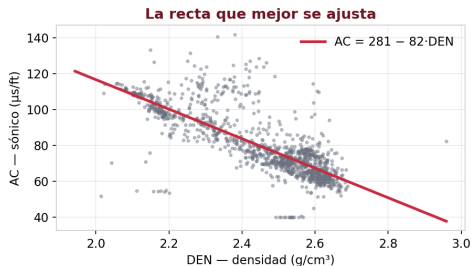
Predecir un valor nuevo:

Python

```
modelo.predict([[2.4]])
```

>_ Output

```
[83.8]
```



¿Qué Tan Bueno es el Modelo?

Las métricas de error

Medir el Error: RMSE y MAE

Un modelo se juzga por su error: qué tan lejos caen las predicciones de la realidad.

 **MAE — Error Absoluto Medio** El promedio de las distancias (sin signo). Fácil de leer: “en promedio me equivoco por tanto”.

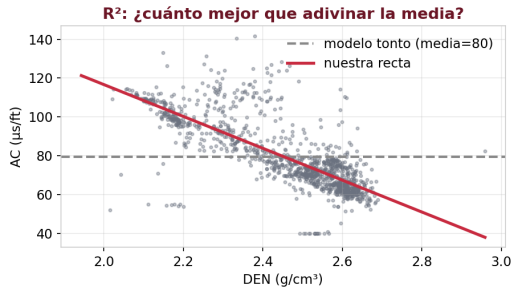
 **RMSE — Raíz del Error Cuadrático** Parecido, pero **castiga más** los errores grandes. Es la métrica más usada.

Ambas están en las **mismas unidades del target** ($\mu\text{s}/\text{ft}$). Entre más **bajas**, mejor. Un RMSE de 0 sería una predicción perfecta.

💡 “RMSE = 11” significa: mis predicciones del sónico se desvían, típicamente, unos 11 $\mu\text{s}/\text{ft}$ del valor real.

R²: ¿Mejor que Adivinar la Media?

RMSE nos da un número, pero ¿11 es bueno o malo? R² lo pone en contexto.



- El “**modelo tonto**” siempre predice la media (la línea gris punteada).
- R² mide cuánto **mejor** es nuestra recta que ese modelo tonto.
- ✓ R² = 1: perfecto. R² = 0: igual que la media. **Se lee como un porcentaje** de lo que el modelo explica.

Las Métricas en Código

sklearn las calcula por nosotros. Este es nuestro modelo simple (AC vs DEN).

Python

```
from sklearn.metrics import (
    mean_squared_error, r2_score)

pred = modelo.predict(X)

rmse = mean_squared_error(y, pred) ** 0.5
r2 = r2_score(y, pred)

print("RMSE:", rmse)
print("R2:", r2)
```

>_ Output

```
RMSE: 11.7
R2: 0.57
```

- ✓ **RMSE 11.7:** nos equivocamos ~ 11.7 $\mu\text{s}/\text{ft}$ en promedio.
- ✓ **R^2 0.57:** la densidad sola explica el **57 %** del sónico.
- ¿El otro 43 %? Otras variables. **Agreguemos más features.**

¿Cuál Métrica Uso? Criterios y Valores Aceptables

Ninguna métrica decide sola: cada una responde una pregunta distinta.

Métrica	Cuándo elegirla	Qué es “aceptable”
MAE	comunicar el error típico en unidades del negocio; robusta a datos raros	júzguela contra la tolerancia de ingeniería del uso final
RMSE	cuando los errores grandes cuestan caro (los castiga más); la estándar para comparar modelos	debe ser mucho menor que la desviación estándar del target (si $RMSE \approx$ desv., el modelo no supera a la media)
R^2	comunicar “% explicado”; comparar entre problemas distintos (no tiene unidades)	> 0.8 excelente · $0.6-0.8$ útil en la industria · $0.4-0.6$ débil · < 0.4 insuficiente

- ✓ Nuestro modelo: R^2 test = **0.66** (útil) y RMSE = **10.6** sobre un target que se mueve entre 40 y 149 (≈ 10 % del rango).
- ⚠ Siempre reportadas **sobre el test**, nunca sobre el train.
- ➔ La decisión final no es estadística: es **¿cuánto cuesta equivocarse** en el uso que le daremos?

De Una a Muchas Variables

Regresión múltiple

Muchas Features: Regresión Múltiple

Una recta con varias features. El código casi no cambia: solo la lista de columnas.

Python

```
X = reg[["DEN", "NEU", "GR", "RDEP"]]
y = reg["AC"]

modelo = LinearRegression()
modelo.fit(X, y)

r2 = modelo.score(X, y)
print("R2:", r2)
```

>_ Output

```
R2: 0.66
```

→ La ecuación ahora tiene **un coeficiente por feature**:

$$AC = b + m_1DEN + m_2NEU + \dots$$

↑ R² sube de **0.57 a 0.66**: juntas explican más que la densidad sola.

¿Cuál Variable Importa Más?

Tentación: mirar los coeficientes y decir “el más grande manda”. Cuidado, es una trampa.

>_ coeficientes crudos

DEN	-55.99
NEU	0.43
GR	0.08
RDEP	-0.06

¿DEN importa 700 veces más que NEU? **No necesariamente.**

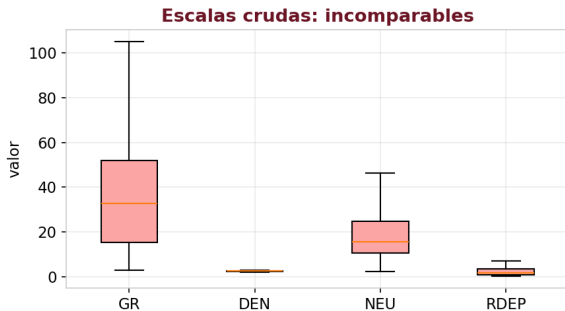
- ⚖️ Los coeficientes dependen de las **unidades**.
- DEN va de 2 a 3 (rango 1); NEU va de 0 a 100 (rango 100).
- Comparar coeficientes de escalas distintas **no es justo**.
- 💡 Solución: poner todas las features en la **misma escala**.

Estandarización

Poner todo en la misma escala

El Problema de las Escalas

Nuestras features viven en mundos numéricos totalmente distintos.



- GR llega a 200, NEU a 100, DEN apenas se mueve entre 2 y 3.
- Para comparar su importancia — y para muchos modelos — esto es un problema.
- 💡 **Estandarizar:** reescalar todas para que sean comparables.

El z-score y StandardScaler

Estandarizar = a cada valor, restarle la media y dividir por la desviación. sklearn lo hace.

Python

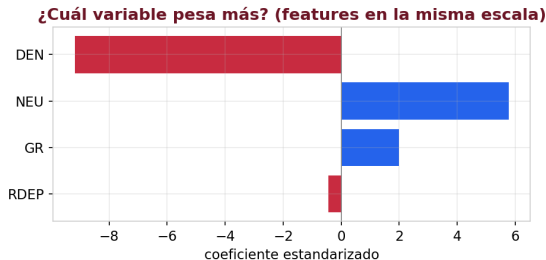
```
from sklearn.preprocessing import  
    StandardScaler  
  
scaler = StandardScaler()  
X_esc = scaler.fit_transform(X)
```

$$z = \frac{\text{valor} - \text{media}}{\text{desviación}}$$

- Después, cada feature tiene **media 0** y **desviación 1**.
- Un valor de +2 significa “2 desviaciones por encima del promedio”, da igual la unidad original.
- ✓ Ya son **comparables**.

Ahora Sí: ¿Cuál Variable Importa Más?

Con las features en la misma escala, los coeficientes ya se pueden comparar.



- 🏆 DEN es la más influyente, seguida de NEU. Coincide con la física.
- ➔ RDEP casi no aporta — como anticipó la correlación.
- ➔ Estandarizar no cambió el R^2 ; cambió nuestra **capacidad de interpretar**.

Una Trampa Silenciosa: Data Leakage

Al estandarizar hay un error clásico que arruina la evaluación sin avisar.



Mal: estandarizar usando **todos** los datos, y luego separar en entrenamiento y prueba.
El scaler “vio” los datos de prueba → información filtrada → resultados demasiado optimistas.



Bien: **primero** separar, y ajustar el scaler **solo** con el entrenamiento.
El conjunto de prueba debe ser como datos “del futuro” que el modelo nunca vio.

💡 Esto nos lleva a la idea más importante del día: **separar entrenamiento y prueba.**

Train / Test Split

¿Cómo sé si mi modelo sirve de verdad?

Una Pregunta Incómoda

Medimos el R^2 sobre los mismos datos con los que entrenamos. ¿Es eso confiable?

*Es como darle a un estudiante el examen **con las respuestas**, dejarlo memorizarlas, y luego evaluarlo con **el mismo examen**. Sacará 100...pero ¿de verdad aprendió?*

- Lo que importa es cómo predice sobre datos **que nunca vio**.
- 💡 Solución: **esconder una parte** de los datos antes de entrenar, y evaluar solo ahí.

train_test_split

Partimos los datos: la mayoría para entrenar, una parte reservada para evaluar.

Python

```
from sklearn.model_selection import
    train_test_split

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=42)

modelo.fit(X_tr, y_tr)           # entrenar
print(modelo.score(X_te, y_te)) # evaluar
```

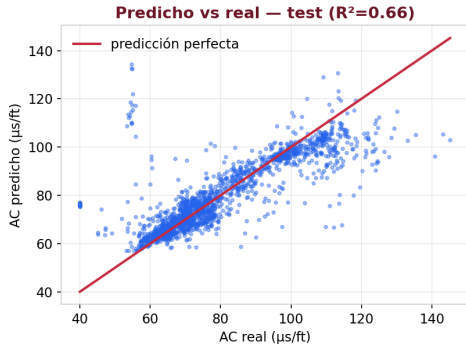
>_ Output

0.66

- `test_size=0.25`: 75 % entrena, 25 % se reserva.
- `random_state`: fija el sorteo para que sea **reproducible**.
- ✓ Entrenamos con `tr`, evaluamos con `te`: datos que el modelo **nunca** vio.

El Veredicto: Predicho vs Real en el Test

La prueba de fuego: comparar predicciones contra la realidad en datos nuevos.



- Cada punto: un tramo del pozo que el modelo **no usó** para entrenar.
- ✓ Caen cerca de la línea roja (predicción perfecta): el modelo **generaliza**.
- R^2 en test $\approx R^2$ en train (0.66 vs 0.67): **no memorizó**, aprendió.

Overfitting: Cuando el Modelo “Memoriza”

El síntoma se lee comparando el desempeño en entrenamiento vs en prueba.

Situación	R^2 train	R^2 test	Diagnóstico
Sano	0.67	0.66	Generaliza bien ✓
Overfitting	0.98	0.55	Memorizó, no aprendió
Underfitting	0.30	0.29	Demasiado simple

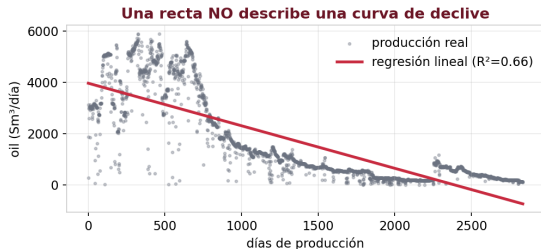
- **Overfitting**: excelente en train, malo en test. El modelo memorizó el ruido en vez de la tendencia.
- ✓ Nuestro modelo está **sano**: por eso el train/test es imprescindible — sin él no lo sabríamos.

Los Límites de la Recta

Qué asume el modelo y cuándo se queda corto

Cuándo la Recta se Queda Corta

Ejemplo real: predecir la producción de un pozo a lo largo del tiempo.



La curva de declive (Clase 3) no es una recta: la línea hasta predice producción **negativa**.

La regresión lineal asume:

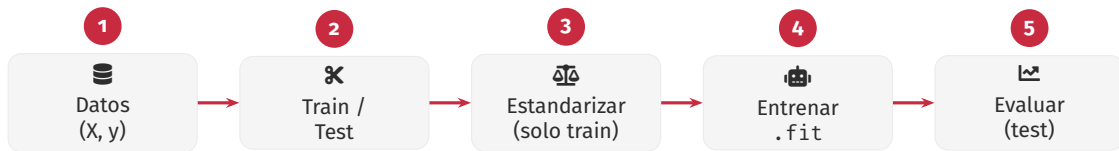
- ✗ **Linealidad:** el patrón es una recta.
- ✗ **Aditividad:** cada feature suma su efecto, sin que unas dependan de otras.
- ✗ **Sensible a outliers:** un dato raro tuerce la recta.

Cuando el patrón es curvo o complejo, se usan modelos **más flexibles** — árboles, bosques, redes neuronales. **Los veremos en las próximas clases del módulo.**

Cierre

Acaban de hacer Machine Learning

El Flujo que Aprendieron Hoy



- ✓ Este flujo es **el mismo** para casi todos los modelos del módulo: hoy fue regresión lineal, la semana entrante serán otros.
- ➔ Cambia el modelo (LinearRegression → otro), pero los pasos se mantienen.

Lo Que Aprendimos Hoy

Conceptos

- ✓ Describir vs **predecir**; qué es un modelo
- ✓ Target y features ($X \rightarrow y$)
- ✓ Regresión lineal: la mejor recta (residuos)
- ✓ Métricas: RMSE, MAE, R^2
- ✓ Correlación, estandarización, data leakage
- ✓ Train/test y overfitting
- ✓ Los límites de la recta (linealidad)

Herramientas (sklearn)

- ✗ LinearRegression — .fit, .predict, .score
- ✗ train_test_split
- ✗ StandardScaler
- ✗ r2_score, mean_squared_error
- ✗ df.corr() para explorar

Logro del día: reconstruir un registro de pozo que costaría re-perforar para medir.

¡Gracias!

Carlos Enrique Mosquera Trujillo

✉ cmosquerat@unal.edu.co

Machine Learning for Petroleum Engineers Using Python

SLB Ecuador · UDLA · 2026